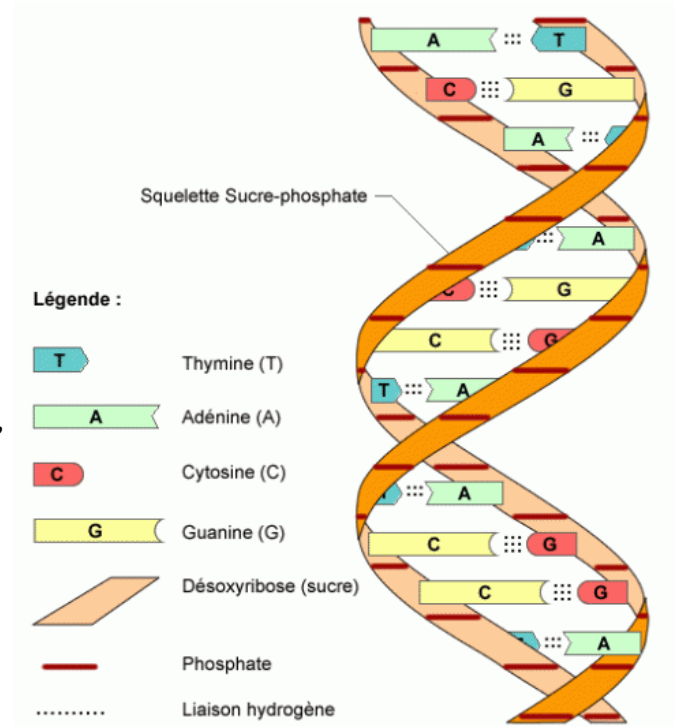


Algorithmes de recherche textuelle

La recherche textuelle consiste en la recherche des occurrences d'une suite de caractères, appelée le motif, dans une autre suite de caractères plus longue, appelée le texte. Les applications sont nombreuses allant de la simple recherche d'un mot dans un document, de mots clés dans une multitude de pages web, jusqu'à la recherche de l'enchaînement des bases nucléiques dans les molécules d'ADN.



I. Utilisation de la force brute

On considère le texte ci-dessous correspondant à un enchaînement de bases nucléiques.

ATAACAGAGAATAAGGCTAGGAAATACTGAA

On souhaite vérifier la présence du motif **AGGCTA**.

La méthode brute consiste à parcourir le texte caractère par caractère tout en vérifiant, à partir de ce caractère, si les caractères suivants correspondent au motif.

1. Enregistrer dans un fichier au format texte **UTF-8** l'enchaînement bases nucléiques.
2. Dans un programme Python nommé **recherche_textuelle_brute.py**, écrire une fonction **ouvrir_fichier(nom_de_fichier)** permettant d'importer le texte.
3. Vérifier avec le mot clé dédié de Python que le motif est bien présent dans le texte.

On considère d'abord le cas d'une recherche dans un texte simple, par exemple que des **A**, contenant un motif d'un seul caractère comme **E**, qui sera déplacé dans le texte pour les tests, notamment au début et à la fin.

4. Ecrire la fonction **recherche_force_brute(texte, motif)** renvoyant la position du motif si le motif est présent dans le texte, **None** sinon.

On se place maintenant dans le cas où le motif est constitué de plusieurs caractères identiques.

5. Modifier la fonction **recherche_force_brute(texte, motif)** pour quelle renvoie la position du premier caractère du motif si le motif est présent dans le texte, **None** sinon.

6. Si ce n'est déjà fait, factoriser cette fonction.

7. Tester votre fonction sur le cas étudié.

8. Combien de tests ont été nécessaires pour trouver le motif dans le texte ?

II. Algorithme de Boyer-Moore

L'algorithme de Boyer-Moore est un algorithme de recherche de sous-chaîne particulièrement efficace. Il a été développé par Robert S. Boyer et J Strother Moore en 1977. L'algorithme pré-traite le motif, et non pas le texte, à l'inverse de certains algorithmes, qui amortissent le coût du prétraitement du texte en effectuant de très nombreuses recherches répétitives (Wikipedia).

1. Appliquer l'algorithme de Boyer-Moore à la recherche du motif **DAB** dans la chaîne de caractère **ABRACADABRA**. Donner toutes les étapes et le nombre de comparaisons effectuées par rapport à la recherche par force brute. Exprimer ce gain en pourcentage.
2. Dans un programme **recherche_textuelle_boyer_moore.py**, reprendre les trois premières questions du I.
3. Ecrire la fonction **table_de_sauts(motif)** qui produit un dictionnaire donnant la distance de chaque nouveau caractère d'un motif par rapport à son dernier caractère.
4. Ecrire une fonction **distance(table, caractere)** qui renvoie la valeur du saut à effectuer pour le caractère donné.
5. **INUTILE modifier la fonction force brute directement** Ecrire une fonction **compare_texte_motif(texte, motif, position)** qui compare le motif au texte texte à partir d'une position en commençant par la fin. La fonction renvoie l'indice du premier caractère non concordant. Si tous les caractères concordent, la fonction renvoie **None**.
6. Ecrire la fonction **recherche_boyer_moore(texte, motif)** qui effectue la recherche de Boyer-Moore. Elle doit renvoyer la position du premier caractère du motif si le motif est présent dans le texte, **None** sinon.
7. Combien de tests ont été nécessaires pour trouver le motif dans le texte ?
8. Conclure sur l'efficacité de cette méthode.

III. Algorithme de Boyer-Moore-Horspool

C'est une version simplifiée de l'algorithme de Boyer-Moore. Nigel Horspool est né en Grande-Bretagne mais est citoyen canadien. Il est professeur émérite d'informatique de l'université de Victoria, retraité depuis 2016.